Contrôle des connaissances de programmation en Python

L1 Informatique – Mathématiques

Durée : 1 h 20

Exercice 1 — PGCD et PPCM

1.1 Écrire une fonction qui prend en argument une liste `L` d'entiers et un entier `x`, et qui vérifie si `x` appartient à `L`. Elle renvoie vrai si `x` est dans `L`, et faux sinon.

1.2 Écrire une fonction qui reçoit une liste en argument et qui renvoie une nouvelle liste ne contenant aucun doublon. Utiliser la fonction de la question 1.1.

Exemple : `sans_doublons([1, 3, 7, 3, 9, 1, 1, 10])` → `[1, 3, 7, 9, 10]`

1.3 Écrire une fonction qui reçoit deux listes **sans doublons** et qui renvoie une liste contenant les entiers appartenant aux deux listes en même temps (intersection). Utiliser la fonction de la question 1.1.

Exemple :
`L1 = [1, 3, 7, 9, 4] ; L2 = [3, 9, 11, 6, 7]`
`inter(L1, L2)` → `[3, 7, 9]`

1.4 Écrire une fonction qui reçoit un entier `n` et qui renvoie une liste contenant tous ses diviseurs.

1.5 Écrire une fonction qui reçoit une liste d'entiers en argument et qui renvoie le plus grand élément.

1.6 Le PGCD (plus grand diviseur commun) de deux entiers est le plus grand entier qui divise les deux. Écrire une fonction `pgcd` en utilisant les fonctions des questions 1.3, 1.4 et 1.5.

1.7 Écrire une fonction qui reçoit deux entiers `n` et `m` et qui renvoie une liste contenant tous les multiples de `n` jusqu'à la valeur `m`.

1.8 Écrire une fonction qui reçoit une liste d'entiers en argument et qui renvoie le plus petit élément.

1.9 Le PPCM (plus petit multiple commun) de deux entiers `n` et `p` est le plus petit entier qui est multiple de `n` et de `p`. En utilisant les fonctions des questions 1.3, 1.7 et 1.8, écrire la fonction `ppcm`. *(Indication : $m = n \times p$ est un multiple de n et de p.)*

Exercice 2 — Fusion de deux listes triées avec contraintes

On considère deux listes d'entiers **triées dans l'ordre croissant** : `L1` et `L2`.

On souhaite fusionner ces deux listes en une seule, **toujours triée**, mais **avec les contraintes suivantes** :

1. **Un seul parcours** des listes :
   - vous n'avez pas le droit de réutiliser `sorted()` ou toute autre méthode de tri ;
   - vous ne pouvez pas parcourir plusieurs fois `L1` ou `L2` ;
   - vous devez avancer progressivement dans les deux listes.
2. **Aucune nouvelle liste créée** :
   - la fusion doit se faire **en modifiant l'une des deux listes existantes** (au choix) ;
   - l'utilisation d'une troisième liste intermédiaire est interdite.

Vous pouvez utiliser : `append` (ajoute à la fin), `insert` (insère à un indice donné) et `len` (calcule la taille).

Rappel : `L1.insert(i, x)` permet d'insérer `x` dans `L1` à l'indice `i` en décalant tous les éléments après `i` (`i` inclus) vers la droite.

Exemple :

```
L1 = [1, 4, 6, 10]
L2 = [2, 3, 5, 7, 8]
fusion_en_place(L1, L2)
print(L1)  # Résultat : [1, 2, 3, 4, 5, 6, 7, 8, 10]
```